# გაკვეთილი 24 mod 3 (3.4)

# გაკვეთილი 3 — დოკუმენტთან სამუშაო ფუნქციები (DOM)

## 1. რა არის DOM?

DOM (Document Object Model) არის HTML დოკუმენტის ობიექტური წარმოდგენა.

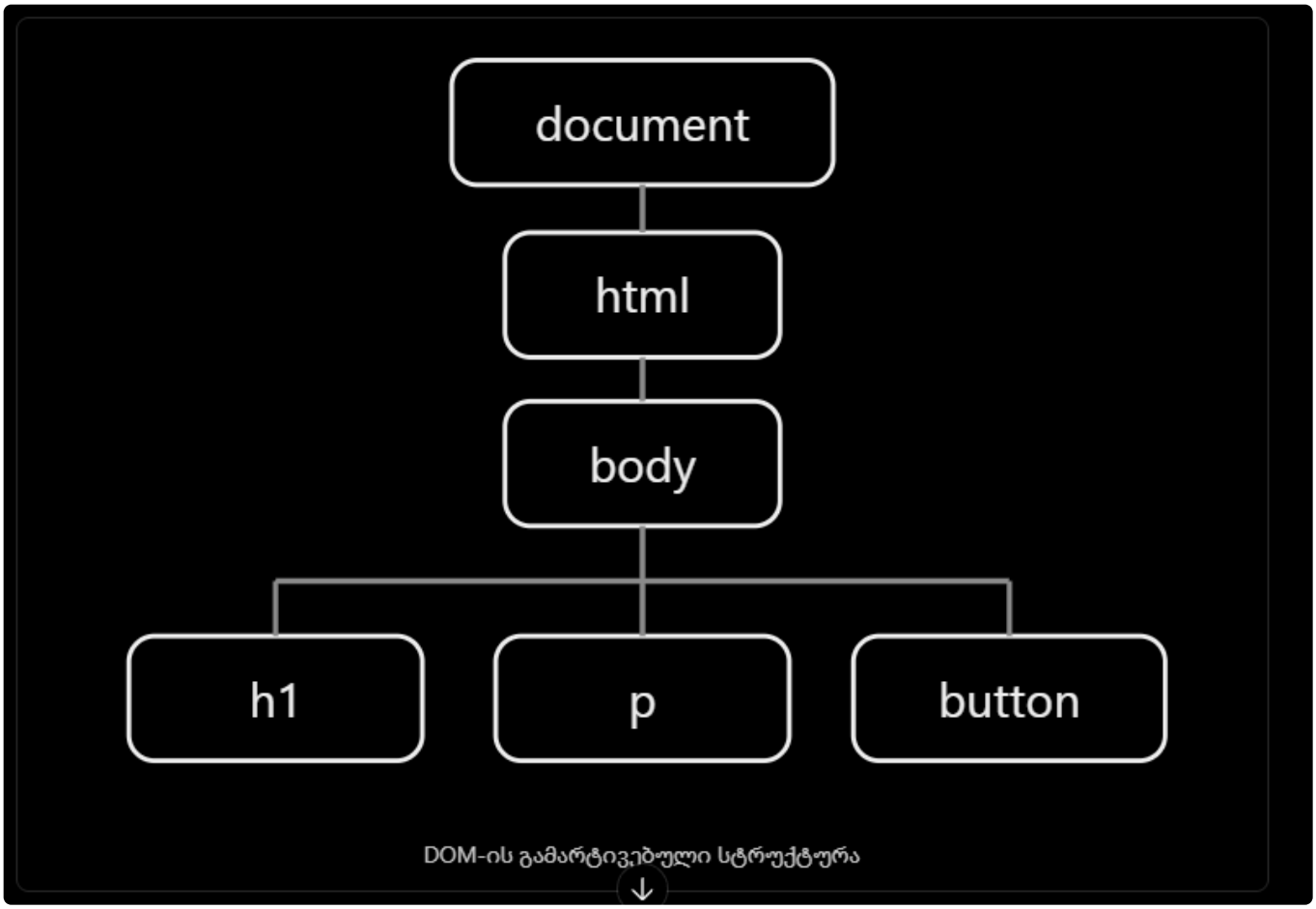
როდესაც ბრაუზერი HTML ფაილს ხსნის, ის ქმნის დოკუმენტის სტრუქტურას, სადაც თითოეული HTML ელემენტი წარმოდგენილია ობიექტის სახით.

მაგალითად, გვაქვს HTML:

```
<html>
<body>
    <h1>Hello World</h1>
    <p>Welcome to my website</p>
    <button>Click Me</button>
</body>
</html>
```

ბრაუზერი ამ დოკუმენტს დაახლოებით ასეთი ხის სახით აღიქვამს:

DOM-ის გამარტივებული სტრუქტურა




JavaScript-ის საშუალებით შეგვიძლია ამ სტრუქტურაში არსებული ელემენტები მოვძებნოთ და შევცვალოთ.

მაგალითად:

- შევცვალოთ სათაურის ტექსტი;
- შევუცვალოთ აბზაცს ფერი;
- დავამატოთ ახალი ღილაკი;
- წავშალოთ რომელიმე ელემენტი;
- მომხმარებლის მოქმედებაზე შევცვალოთ გვერდის შიგთავსი.

მნიშვნელოვანი: DOM-თან მუშაობისას HTML ფაილის საწყისი კოდი არ იცვლება. JavaScript ცვლის ბრაუზერში არსებულ დოკუმენტს.

## 2. `document` ობიექტი

`document` არის ობიექტი, რომელიც წარმოადგენს ბრაუზერში გახსნილ HTML დოკუმენტს.

მაგალითად:

```
console.log(document);
```

ამ კოდის შესრულებისას Console-ში გამოჩნდება HTML დოკუმენტის სტრუქტურა.

შეგვიძლია მივიღოთ დოკუმენტის სათაურიც:

```
console.log(document.title);
```

თუ HTML-ში გვაქვს:

```
<title>My Website</title>
```

შედეგი იქნება:

```
My Website
```

შეგვიძლია სათაურის შეცვლაც:

```
document.title = "New Website";
```

ამ შემთხვევაში ბრაუზერის ჩანართის სათაური შეიცვლება.

# 3. HTML ელემენტების მოძებნა

JavaScript-ით ელემენტის შესაცვლელად ჯერ უნდა მოვძებნოთ იგი.

ამისთვის რამდენიმე მეთოდი არსებობს.

## `getElementById()`

ეს მეთოდი პოულობს HTML ელემენტს მისი `id` ატრიბუტის მიხედვით.

მაგალითად:

```
<h1 id="title">Hello World</h1>
```

JavaScript:

```
let heading = document.getElementById("title");

console.log(heading);
```

შედეგად მივიღებთ შესაბამის `h1` ელემენტს.

აქ:

- `document` — HTML დოკუმენტია;
- `getElementById()` — ელემენტის მოძებნის მეთოდია;
- `"title"` — ელემენტის ID.

თუ მითითებული ID არ არსებობს, მეთოდი აბრუნებს `null`-ს.

## `getElementsByClassName()`

ეს მეთოდი პოულობს ელემენტებს მათი `class` ატრიბუტის მიხედვით.

მაგალითად:

```
<p class="text">First paragraph</p>
<p class="text">Second paragraph</p>
<p class="text">Third paragraph</p>
```

JavaScript:

```
let paragraphs = document.getElementsByClassName("text");

console.log(paragraphs);
```

ეს მეთოდი აბრუნებს HTMLCollection-ს — ელემენტების კოლექციას.

შეგვიძლია კონკრეტულ ელემენტს ინდექსით მივწვდეთ:

```
console.log(paragraphs[0]);
console.log(paragraphs[1]);
```

პირველი ელემენტის ინდექსია `0`, როგორც მასივებში.

## `querySelector()`

`querySelector()` პოულობს პირველ ელემენტს, რომელიც მითითებულ CSS სელექტორს შეესაბამება.

მაგალითად:

```
<h1 id="title">Hello</h1>
```

```
<p class="text">Welcome</p>
```

JavaScript:

```
let heading = document.querySelector("#title");

let paragraph = document.querySelector(".text");
```

აქ CSS-დან უკვე ნაცნობი წესები გამოიყენება:

- `#title` — ID;
- `.text` — Class;
- `p` — HTML ტეგი.

მაგალითად:

```
let paragraph = document.querySelector("p");
```

იპოვის პირველ `p` ელემენტს.

## `querySelectorAll()`

`querySelectorAll()` პოულობს ყველა ელემენტს, რომელიც მითითებულ სელექტორს შეესაბამება.

```
<p class="text">HTML</p>
<p class="text">CSS</p>
<p class="text">JavaScript</p>
```

```
let items = document.querySelectorAll(".text");

console.log(items);
```

შედეგად მივიღებთ NodeList-ს, რომელშიც სამი ელემენტია.

მაგალითად, ყველა ელემენტის ტექსტის გამოსატანად:

```
let items = document.querySelectorAll(".text");

items.forEach(function(item) {
    console.log(item.textContent);
});
```

შედეგი:

```
HTML
CSS
JavaScript
```

### მეთოდების შედარება

| მეთოდი | რას აბრუნებს |
|---|---|
| `getElementById()` | ერთ ელემენტს ან `null`-ს |
| `getElementsByClassName()` | HTMLCollection-ს |
| `querySelector()` | პირველ შესაბამის ელემენტს ან `null`-ს |
| `querySelectorAll()` | NodeList-ს ყველა შესაბამისი ელემენტით |

## 4. HTML ელემენტის ტექსტის შეცვლა

ახლა, როდესაც ელემენტის მოძებნა ვიცით, შეგვიძლია მისი ტექსტიც შევცვალოთ.

### `textContent`

`textContent` გამოიყენება ელემენტის ტექსტური შიგთავსის მისაღებად ან შესაცვლელად.

მაგალითად:

```
<h1 id="title">Hello World</h1>
```

JavaScript:

```
let title = document.getElementById("title");

title.textContent = "Welcome to JavaScript";
```

შედეგად გვერდზე გამოჩნდება:

```
Welcome to JavaScript
```

ეს ნიშნავს, რომ JavaScript-მა HTML ელემენტის ტექსტი შეცვალა.

### `innerHTML`

`innerHTML` საშუალებას გვაძლევს ელემენტის შიგნით HTML კოდიც ჩავწეროთ.

მაგალითად:

```
<div id="content"></div>
```

JavaScript:

```
let content = document.getElementById("content");

content.innerHTML = "<h2>Hello!</h2><p>Welcome!</p>";
```

შედეგად `div`-ის შიგნით შეიქმნება სათაური და აბზაცი.

განსხვავება:

`textContent` ტექსტს ჩვეულებრივ ტექსტად აღიქვამს, ხოლო `innerHTML` მას HTML კოდად ამუშავებს.

მაგალითად:

```
title.textContent = "<b>Hello</b>";
```

გამოიტანს თვითონ ტექსტს `<b>Hello</b>`.

ხოლო:

```
title.innerHTML = "<b>Hello</b>";
```

გამოიტანს გამუქებულ Hello-ს.

მნიშვნელოვანია: მომხმარებლის მიერ შეყვანილი ან სხვა არასანდო ტექსტი პირდაპირ `innerHTML`-ში არ უნდა ჩავსვათ, რადგან ამან შეიძლება უსაფრთხოების პრობლემა შექმნას. ასეთი ტექსტისთვის გამოიყენეთ `textContent`.

# 5. CSS სტილის შეცვლა JavaScript-ით

JavaScript-ს შეუძლია HTML ელემენტის სტილის შეცვლაც.

ამისთვის ვიყენებთ `style` თვისებას.

მაგალითად:

```
<h1 id="title">Hello World</h1>
```

JavaScript:

```
let title = document.getElementById("title");

title.style.color = "red";
```

სათაურის ფერი გახდება წითელი.

შეგვიძლია სხვა სტილებიც შევცვალოთ:

```
title.style.backgroundColor = "yellow";
title.style.fontSize = "30px";
title.style.textAlign = "center";
```

აქ ყურადღება მიაქციეთ:

CSS-ში ვწერთ:

```
background-color: yellow;
font-size: 30px;
```

JavaScript-ში კი:

```
backgroundColor
fontSize
```

ანუ გამოიყენება `camelCase`.

## სრული მაგალითი

```
<!DOCTYPE html>
<html>
<head>
    <title>Style Example</title>
</head>
<body>

    <h1 id="title">Hello World</h1>

    <button onclick="changeStyle()">Change Style</button>
```

```
    <script>
        function changeStyle() {
            let title = document.getElementById("title");

            title.style.color = "blue";
            title.style.fontSize = "40px";
            title.style.textAlign = "center";
        }
    </script>

</body>
</html>
```

ღილაკზე დაჭერის შემდეგ სათაური შეიცვლის ფერს, ზომას და განლაგებას.

ინტერაქტიული მაგალითი

## Hello World

Change Styleდააჭირეთ ღილაკს და ნახეთ, როგორ იცვლება სათაურის სტილი.

# 6. HTML ელემენტის ატრიბუტების შეცვლა

HTML ელემენტებს სხვადასხვა ატრიბუტი აქვთ.

მაგალითად:

```
<img id="photo" src="image1.jpg" alt="Photo">
```

JavaScript-ით შეგვიძლია `src` ატრიბუტის შეცვლა:

```
let image = document.getElementById("photo");

image.src = "image2.jpg";
```

ან გამოვიყენოთ `setAttribute()`:

```
image.setAttribute("src", "image2.jpg");
```

ორივე შემთხვევაში სურათის მისამართი შეიცვლება.

ატრიბუტის წაკითხვაც შეგვიძლია:

```
console.log(image.getAttribute("src"));
```

# 7. ახალი HTML ელემენტის შექმნა

ზოგჯერ გვინდა JavaScript-ით სრულიად ახალი HTML ელემენტი შევქმნათ.

ამისთვის გამოიყენება:

```
document.createElement()
```

მაგალითად:

```
let paragraph = document.createElement("p");
```

ეს ქმნის ახალ `p` ელემენტს, მაგრამ ჯერ გვერდზე არ აჩვენებს.

ტექსტის დამატება:

```
paragraph.textContent = "This is a new paragraph";
```

ახლა ელემენტი უნდა დავამატოთ HTML დოკუმენტში.

ამისთვის გამოიყენება `appendChild()`:

```
document.body.appendChild(paragraph);
```

შედეგად გვერდზე გამოჩნდება ახალი აბზაცი.

## სრული მაგალითი

```
<!DOCTYPE html>
<html>
<head>
    <title>Create Element</title>
</head>
<body>

    <h1>My Website</h1>

    <button onclick="addParagraph()">Add Paragraph</button>

    <script>
```

```
        function addParagraph() {
            let paragraph = document.createElement("p");

            paragraph.textContent = "New paragraph added!";

            document.body.appendChild(paragraph);
        }
    </script>

</body>
</html>
```

ყოველ დაჭერაზე შეიქმნება ახალი აბზაცი.

# 8. HTML ელემენტის წაშლა

თუ ელემენტის დამატება შეგვიძლია, შეგვიძლია მისი წაშლაც.

ამისთვის გამოიყენება:

```
remove()
```

მაგალითად:

```
<p id="message">Delete me!</p>
```

JavaScript:

```
let message = document.getElementById("message");

message.remove();
```

შედეგად აბზაცი გაქრება გვერდიდან.

## სრული მაგალითი

```
<p id="message">Hello World</p>

<button onclick="deleteText()">Delete</button>

<script>
    function deleteText() {
        let message = document.getElementById("message");
```

```
        if (message) {
            message.remove();
        }
    }
</script>
```

`if (message)` ამოწმებს, არსებობს თუ არა ელემენტი, რათა მისი წაშლის მცდელობამ შეცდომა არ გამოიწვიოს.

# 9. მოვლენები (Events) და DOM

JavaScript-ში ხშირად გვჭირდება მოქმედების შესრულება მომხმარებლის ქცევის მიხედვით.

მაგალითად:

- ღილაკზე დაჭერა;
- ტექსტის შეყვანა;
- ფორმის გაგზავნა;
- ელემენტზე მაუსის გადატანა.

ასეთ მოქმედებებს Events ეწოდება.

ამ გაკვეთილში განვიხილავთ `click` მოვლენას.

## `addEventListener()`

```
<button id="btn">Click Me</button>
```

JavaScript:

```
let button = document.getElementById("btn");

button.addEventListener("click", function() {
    alert("Button clicked!");
});
```

აქ:

- `button` — მოძებნილი ღილაკია;
- `addEventListener()` — მოვლენაზე რეაგირების მეთოდია;
- `"click"` — ღილაკზე დაჭერის მოვლენაა;

- `function()` — ფუნქცია, რომელიც დაჭერისას შესრულდება.

ეს არის `onclick`-ის ალტერნატივა და ხშირად უფრო მოსახერხებელია, რადგან JavaScript კოდს HTML-ისგან ცალკე ვტოვებთ.

## 10. სრული პრაქტიკული მაგალითი — ტექსტის მართვა

შევქმნათ ვებგვერდი, სადაც მომხმარებელს შეუძლია ტექსტის შეცვლა, ფერის შეცვლა, ახალი ელემენტის დამატება და წაშლა.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>DOM Practice</title>
</head>
<body>

    <h1 id="title">Hello World</h1>

    <button onclick="changeText()">Change Text</button>
    <button onclick="changeColor()">Change Color</button>
    <button onclick="addText()">Add Text</button>
    <button onclick="removeText()">Remove Text</button>

    <div id="content"></div>

    <script>
        function changeText() {
            let title = document.getElementById("title");

            title.textContent = "JavaScript DOM";
        }

        function changeColor() {
            let title = document.getElementById("title");

            title.style.color = "blue";
        }

        function addText() {
            let paragraph = document.createElement("p");

            paragraph.textContent = "New paragraph!";
```

```
            document.getElementById("content").appendChild(paragraph);
        }

        function removeText() {
            let content = document.getElementById("content");

            if (content.lastElementChild) {
                content.lastElementChild.remove();
            }
        }
    </script>

</body>
</html>
```

როგორ იმუშავებს პროგრამა




ამ მაგალითში მოსწავლეები პრაქტიკულად ხედავენ, როგორ მუშაობს DOM და როგორ ცვლის JavaScript ვებგვერდის შიგთავსს.

# პრაქტიკული სავარჯიშოები

## სავარჯიშო 1 — სათაურის შეცვლა (მარტივი)

მოცემულია:

```
<h1 id="title">Hello</h1>
<button onclick="changeTitle()">Change</button>
```

დაწერეთ JavaScript ფუნქცია `changeTitle()`, რომელიც ღილაკზე დაჭერისას შეცვლის სათაურის ტექსტს:

```
Welcome to JavaScript!
```

გამოიყენეთ `getElementById()` და `textContent`.

## სავარჯიშო 2 — ფერისა და ზომის შეცვლა (მარტივი)

მოცემულია:

```
<p id="text">Learning JavaScript is fun!</p>

<button onclick="changeStyle()">Change Style</button>
```

შექმენით ფუნქცია, რომელიც ღილაკზე დაჭერისას:

1. ტექსტის ფერს შეცვლის წითლად;
2. ტექსტის ზომას გახდის `25px`;
3. ფონის ფერს გახდის ყვითელს.

გამოიყენეთ `style`.

## სავარჯიშო 3 — ყველა აბზაცის შეცვლა (საშუალო)

მოცემულია:

```
<p class="item">HTML</p>
<p class="item">CSS</p>
<p class="item">JavaScript</p>

<button onclick="changeAll()">Change All</button>
```

დაწერეთ ფუნქცია, რომელიც ღილაკზე დაჭერისას სამივე აბზაცის ფერს ლურჯად შეცვლის.

გამოიყენეთ `querySelectorAll()` და `forEach()`.

## სავარჯიშო 4 — ახალი ელემენტის დამატება (საშუალო)

შექმენით HTML გვერდი, რომელზეც იქნება:

```
<div id="container"></div>

<button onclick="addItem()">Add Item</button>
```

ღილაკზე ყოველი დაჭერისას `container`-ში უნდა დაემატოს ახალი აბზაცი ტექსტით:

```
New Item
```

გამოიყენეთ:

```
createElement()
textContent
appendChild()
```

## სავარჯიშო 5 — ელემენტის წაშლა (საშუალო)

მოცემულია:

```
<h2 id="message">This message will disappear</h2>

<button onclick="removeMessage()">Remove</button>
```

დაწერეთ ფუნქცია, რომელიც ღილაკზე დაჭერისას წაშლის `h2` ელემენტს.

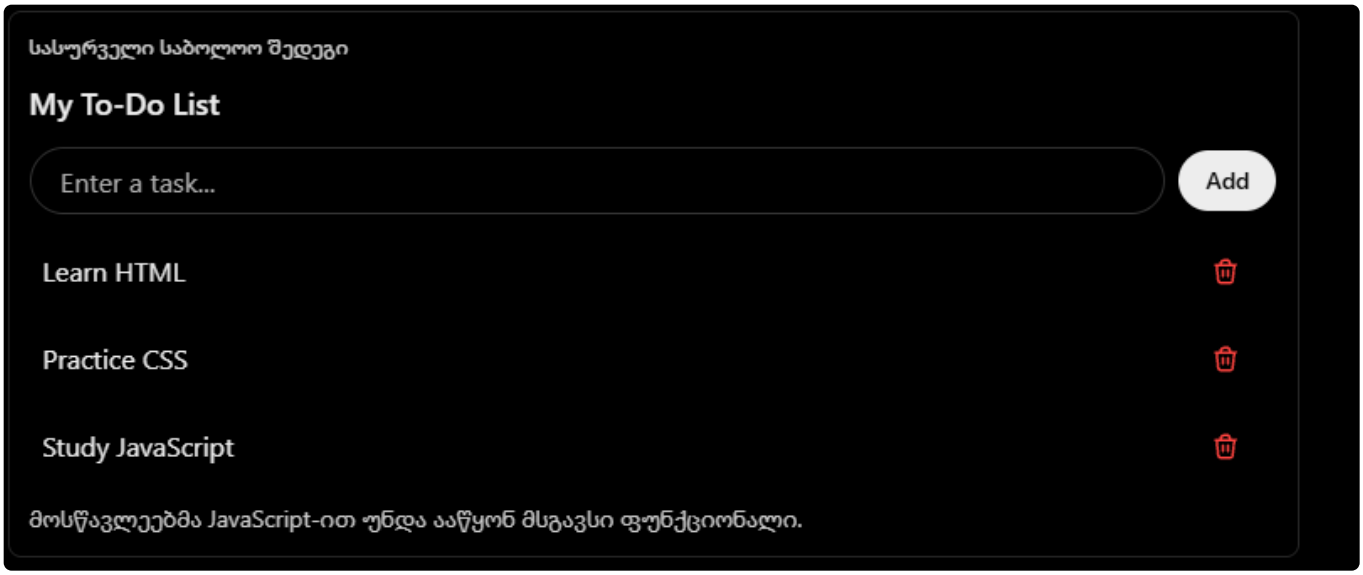
გამოიყენეთ `getElementById()` და `remove()`.

# შემაჯამებელი პრაქტიკული დავალება — Interactive To-Do List

შედარებით რთული დავალება

შექმენით პატარა To-Do List, რომელშიც მომხმარებელს შეეძლება ახალი დავალების დამატება და წაშლა.

სასურველი საბოლოო შედეგი




## მოთხოვნები

1. HTML-ში შექმენით `input`, `button` და ცარიელი `ul` ელემენტი.
2. მომხმარებელმა `input`-ში უნდა ჩაწეროს დავალების დასახელება.
3. Add ღილაკზე დაჭერისას უნდა შეიქმნას ახალი `li` ელემენტი.
4. `li`-ში უნდა გამოჩნდეს მომხმარებლის მიერ შეყვანილი ტექსტი.
5. თითოეულ დავალებას უნდა ჰქონდეს Delete ღილაკი.
6. Delete ღილაკზე დაჭერისას შესაბამისი დავალება უნდა წაიშალოს.

## საწყისი კოდი

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>To-Do List</title>
</head>
<body>

    <h1>My To-Do List</h1>

    <input type="text" id="taskInput" placeholder="Enter task">

    <button onclick="addTask()">Add Task</button>

    <ul id="taskList"></ul>

    <script>
        function addTask() {
            // დაწერეთ კოდი აქ
        }
    </script>

</body>
</html>
```

მოსწავლეებმა უნდა გამოიყენონ:

- `getElementById()`
- `createElement()`
- `textContent`
- `appendChild()`
- `remove()`

- `addEventListener()`